

# Bryce Tabangcura - Lab Week 2

Lab partner: haven bunter-gooley

## Pre-Lab

1. Equation that relates  $n$ ,  $\theta$ ,  $d$  and  $\lambda$

$$d \sin(\theta) = n\lambda$$

2. Solving for 4000 and 440

$$d \sin(\theta) = n\lambda$$

$$\theta = \arcsin\left(\frac{n\lambda}{d}\right)$$

$$d = \frac{1 \text{ cm}}{4000} = \frac{0.01 \text{ m}}{4000}$$

$$d = 2.5 \times 10^{-6} \text{ m}$$

$$n = 1$$

$$\lambda = 0.44 \times 10^{-6} \text{ m}$$

$$\theta = \arcsin\left(\frac{1 \times 0.44 \times 10^{-6}}{2.5 \times 10^{-6}}\right)$$

$$\theta = \arcsin(0.176)$$

$$\theta \approx 10.14^\circ$$

## Preamble

The entire experiment was recorded and data interpolated in python. Here is simple helper functions and imports.

These can mostly be ignored.

```
import warnings
from math import log, pi, sqrt

import matplotlib.pyplot as plt
import numpy as np
from scipy import constants, odr
```

```

from scipy.stats import norm
from uncertainties import UFloat, ufloat, umath

warnings.filterwarnings("ignore", category=FutureWarning,
module="uncertainties")
warnings.filterwarnings(
    "ignore",
    category=FutureWarning,
    message=r"AffineScalarFunc\.__abs__\(\) is deprecated",
)
warnings.filterwarnings(
    "ignore", category=UserWarning, message="Using UFloat objects with
std_dev==0"
)

def fit_odr(x: list[UFloat], y: list[UFloat], zero_intercept: bool = False):
    x_nom, x_err = [v.n for v in x], [v.s for v in x]
    y_nom, y_err = [v.n for v in y], [v.s for v in y]
    data = odr.RealData(x_nom, y_nom, sx=x_err, sy=y_err)
    if zero_intercept:
        model, beta0 = odr.Model(lambda p, x: p[0] * x), [1]
    else:
        model, beta0 = odr.Model(lambda p, x: p[0] * x + p[1]), [1, 0]
    out = odr.ODR(data, model, beta0=beta0).run()
    slope = ufloat(out.beta[0], out.sd_beta[0])
    intercept = (
        ufloat(out.beta[1], out.sd_beta[1]) if not zero_intercept else
        ufloat(0, 0)
    )
    y_pred = model.fcn(out.beta, np.array(x_nom))
    ss_res = np.sum((np.array(y_nom) - y_pred) ** 2)
    ss_tot = np.sum(
        np.array(y_nom) ** 2
        if zero_intercept
        else (np.array(y_nom) - np.mean(y_nom)) ** 2
    )
    r_squared = 1 - ss_res / ss_tot
    return slope, intercept, r_squared

def compsigma(a, b):
    return abs((a.n - b.n) / (((a.s**2) + (b.s**2)) ** (1 / 2))))

def plot(
    x,
    y,

```

```

        fitodr=None,
        title="default title",
        xlabel="default xlabel",
        ylabel="default ylabel",
        show=True,
    ):
        if fitodr is None:
            fitodr = fit_odr(x, y)
        slope, intercept, r_squared = fitodr
        plt.figure()
        first = x[0].n
        last = x[-1].n

        plt.plot(
            [first, last],
            [first * slope.n + intercept.n, last * slope.n + intercept.n],
            "r-",
            label=f"Fit: $y = ({slope.n:.4g})x + ({intercept.n:.4g}), R^2 = {r_squared:.4f}$",
        )

        plt.errorbar(
            [v.nominal_value for v in x],
            [v.nominal_value for v in y],
            xerr=[v.std_dev for v in x],
            yerr=[v.std_dev for v in y],
            capsize=5,
            linestyle="none",
            marker="o",
            markersize=3,
            label="Data",
        )

        plt.title(title)
        plt.xlabel(xlabel)
        plt.ylabel(ylabel)
        plt.legend()
        if show:
            plt.show()

def plot_residuals(
    x,
    y,
    fitodr=None,
    title="Residuals",
    xlabel="default xlabel",
    ylabel="Residual (y - fit)",

```

```

        show=True,
    ):
        if fitodr is None:
            fitodr = fit_odr(x, y)
            slope, intercept, r_squared = fitodr
            residuals = [yi - (slope * xi + intercept) for xi, yi in zip(x, y)]
            sigma_eff = [
                sqrt(yi.std_dev**2 + (slope.n * xi.std_dev) ** 2) for xi, yi in zip(x,
y)
            ]
            chi2 = sum((r.nominal_value**2) / (s**2) for r, s in zip(residuals,
sigma_eff))
            dof = len(y) - 1
            chi2_reduced = chi2 / dof
            zero_fit = (
                ufloat(0, 0),
                ufloat(0, 0),
                0.0,
            ) # was None - plot()'s label needs a float

        plot(
            x,
            residuals,
            fitodr=zero_fit,
            title=f"{title} ( $\chi^2_{\nu} = \{chi2\_reduced:.3f\}$ )",
            xlabel=xlabel,
            ylabel=ylabel,
            show=show,
        )

    return chi2_reduced

def two_tailed_p(diff: float):
    z = abs(diff)
    return 2 * norm.sf(z)

```

## Experiment 1

### Aim

To experimentally determine some material through the use of a spectrometer.

### Method

At the collimator end of a mass spectrometer, we first place our known mercury source gas emitter lamp.

We then recorded the first angle on both sides for any slit. We will average out the reading to find the center.

We then replace the mercury with an unknown source gas emitter lamp. We record the slits with the brightest slits, so as to choose the first order frequencies, this was done with our own discretion, rather than objective source of reasoning.

We repeat this for all of the unknown source gas emitters.

This was all recorded in the dark.

## Results

The uncertainty in our angles was chosen to be 7.5 minutes. The spectrometer had big notches of minutes as 15 minutes. We attempted to record accurate to each minute, but we choose the uncertainty as half the big notches since the difficulty of reading a spectrometer in the dark.

```
uncertainty = 7.5 / 60
middle = umath.radians( # pyright: ignore[reportAttributeAccessIssue]
    (ufloat(40 + 7 / 60, uncertainty) + ufloat(12 + 11 / 60, uncertainty)) / 2
)
unknown1 = [41 + 49 / 60, 43 + 55 / 60, 47]
unknown2 = [48 + 55 / 60, 48, 47 + 15 / 60]
unknown3 = [47]
```

For the list of common spectral lines, we have python table-ised them to make it easier to do data interpretation

```
spectral_lines = [
    (706.5, "He"),
    (703.2, "Ne"),
    (670.8, "Li"),
    (667.8, "He"),
    (650.6, "Ne"),
    (643.8, "Cd"),
    (640.2, "Ne"),
    (636.2, "Zn"),
    (633.4, "Ne"),
    (610, "Eu"),
    (589.0, "Na"),
    (589.6, "Na"),
    (587.5, "He"),
    (579, "Hg"),
    (576.9, "Hg"),
    (546.1, "Hg"),
    (508.6, "Cd"),
    (501.5, "He"),
    (481, "Zn"),
    (480, "Cd"),
```

```

(472, "Zn"),
(467.8, "Cd"),
(460.3, "Li"),
(447.1, "He"),
(435.8, "Hg"),
(404.6, "Hg"),
(388.8, "He"),
(365.4, "Hg"),
(360, "Cd"),
]

```

Math time  $d \sin \theta = n\lambda$  since we are trying to find the strongest lights, we can assume  $n = 1$ .

our diffraction grating is 600 per mm, so we do 1mm divided by our 600.  $d = 1e-3 / 600$

$\frac{1 \times 10^{-3}}{600 \times 1} \sin \theta = \lambda$  So we do that, we define a calc wavelength function that takes theta as its parameter, which we can reuse and solve for unknown waves

```

def calc_wavelength(x: UFloat):
    D = 1e-3 / 600
    return D * umath.sin(x) # type: ignore

unknown1rad = [ufloat(np.deg2rad(x), np.deg2rad(uncertainty)) for x in
unknown1]
unknown2rad = [ufloat(np.deg2rad(x), np.deg2rad(uncertainty)) for x in
unknown2]
unknown3rad = [ufloat(np.deg2rad(x), np.deg2rad(uncertainty)) for x in
unknown3]

unknown1waves = [calc_wavelength(x - middle) for x in unknown1rad]
unknown2waves = [calc_wavelength(x - middle) for x in unknown2rad]
unknown3waves = [calc_wavelength(x - middle) for x in unknown3rad]
[print(x) for x in unknown1waves]
pass

```

```

(4.50+/-0.04)e-07
(5.09+/-0.04)e-07
(5.93+/-0.04)e-07

```

## Conclusions

Using some python manipulation, we can find the most similarly matching for each wave.

This is programmer pilled but we compsigma to all the spectral lines for each wavelength, we then sort that and pick only the top n.

```
# Just wanted to mention no ai was used ;3
def gettopwaves(unknown):
    return sorted(
        [
            [
                (compsigma(ufloat(spectral[0] / 1e9, 1e-15), wave),
spectral[1])
                for spectral in spectral_lines
            ]
            for wave in unknown
        ]
    )

def getn(unknown, n):
    return [sorted(x, key=lambda x: x[0])[0:n] for x in gettopwaves(unknown)]
```

Simply we can use this function.

```
[print(x) for x in getn(unknown1waves, 3)]
pass
```

```
[(0.8660839744586977, 'Na'), (1.0102566684085588, 'Na'), (1.3706884032831859,
'He')]
[(0.007338510761072613, 'Cd'), (1.6668315716912985, 'He'), (6.500702936518553,
'Zn')]
[(0.692004680825812, 'He'), (2.3864586308877533, 'Li'), (3.3273558491866706,
'Hg')]
```

For unknown one, for each slit we measured, helium appears in the top three matches all three times, so its likely helium.

We will use the function again for 3, we skip 2 and come back later.

```
[print(x) for x in getn(unknown3waves, 3)]
pass
```

```
[(0.8660839744586977, 'Na'), (1.0102566684085588, 'Na'), (1.3706884032831859,
'He')]
```

We observe that Na appears twice, note that Na has two slits in a close position, and that this was the only slit we could see, and that Na also has that property. so we will assume it to be Na or sodium

```
[print(x) for x in getn(unknown2waves, 5)]  
pass
```

```
[(0.2837902583328232, 'Cd'), (1.1604845165233584, 'Ne'), (1.3721877849160085,  
'Ne'), (2.1345892478461836, 'Zn'), (2.81646255977219, 'Ne')]  
[(2.4910814822454217, 'Eu'), (3.170145883954749, 'Ne'), (3.847557705551377,  
'Zn'), (4.815288879260805, 'Ne'), (5.686246935599284, 'Cd')]  
[(2.408181414194925, 'Eu'), (2.501896607498632, 'Na'), (2.6463106669602188,  
'Na'), (3.0073458156141593, 'He'), (5.053211657986481, 'Hg')]
```

We observe no pattern in our unknown waves. Not quantitatively, there appeared to be a multitude of slits at one point, this is likely why we chose the wrong values, we picked the 3 most prominent colours (not qualitatively).

The bunched up slits we observed is similar to the characteristics of neon, and neon appears a multitude of times, and appears the most out of any element. The unknown 2 is likely neon only cause of the bunched up slits we observed.